



3

11

1320 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

List in prolog

عملي مشترك



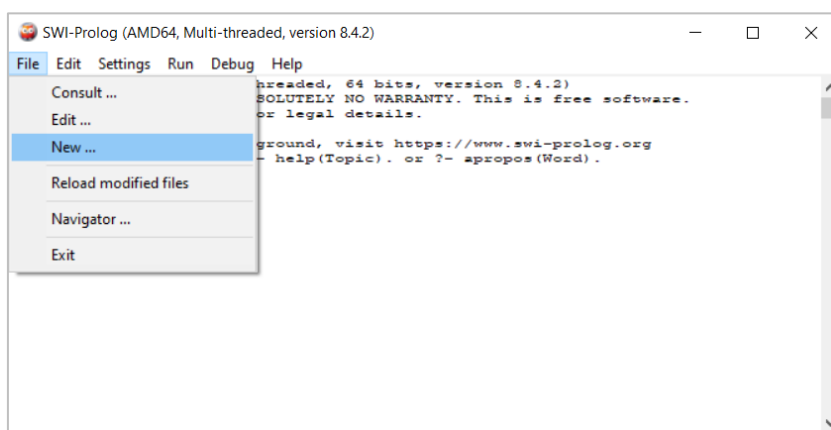
RB Informatcs; 03/05/2026

محتوى مجاني غير مخصص للبيع التجاري

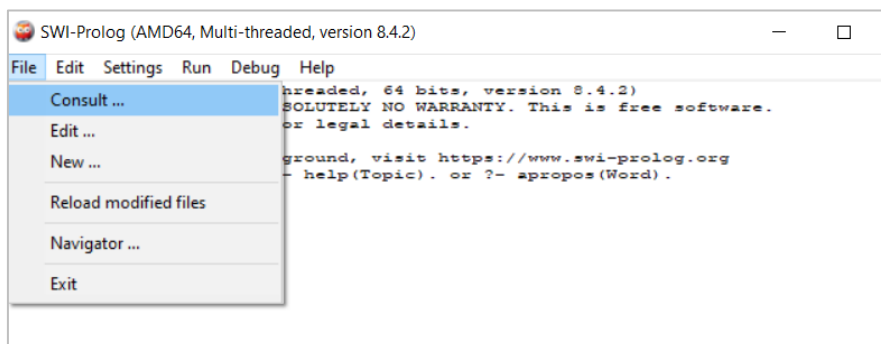
مبادئ الذكاء الصناعي

في هذه المحاضرة سنتحدث مفهوم القوائم (Lists) في لغة البرمجة المنطقية برولوج مع التركيز على بنيتها الأساسية المعتمدة على الرأس (Head) والذيل (Tail) ولكن قبل التعمق بها سنقوم بحل تمرينين عن الأفكار السابقة ومن ثم نتابع:

بداية سنستخدم تطبيق SWI-Prolog ونفتحه ومن القائمة العلوية اختر File ثم New.



نقوم بتسمية الملف مثلاً (task.pl) وهذا الملف سنقوم بكتابة الاكواد داخل نرجع للشاشة البيضاء الأساسية (الكونسول) ونختار File ثم Consult ونختار الملف الذي انشأنه والان اصبح البرنامج يعرف القواعد التي كتبها.



ملاحظة:

أحياناً نكتب كوداً عودياً (Recursion) فيه خطأ منطقي، فيظل البرنامج يبحث للأبد ولا يتوقف، ونلاحظ أن البرنامج (علق) هنا نضغط على الاختصار Ctrl+C من لوحة المفاتيح ثم نكتب abort لتجبر البرنامج على العودة للوضع الطبيعي

تمرين:

➤ اكتب Predicate لحساب العامل لرقم ما:

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce
main.pl
fact(0,1).
fact(X,Res) :-
    X > 0,
    X1 is X - 1,
    fact(X1,R1),
    Res is X * R1.
```

الشرح:

• القاعدة الأولى:

fact(0, 1) : تخبر البرنامج أن يتوقف عن الاستدعاء الذاتي عندما يصل إلى الصفر ويعيد النتيجة 1 مباشرة

• القاعدة الثانية تقول:

إذا X أكبر من 0 ننقصه 1 ونخزنه في X1 ومن ثم نحسب factorial لـ X1 بشكل تكراري

والنتيجة النهائية: result = X * (X-1)

وفي السطر 4: أجرينا عملية الطرح X-1 وباستخدام is أسندنا الناتج للمتغير X1 لتجهيزه للمرحلة التالية.

وأيضاً في السطر 6 قمنا بعملية الضرب X * Result وباستخدام is أسندنا الناتج النهائي للمتغير Result

مثلاً عند هذا الاستدعاء: 5! = 5 × 4 × 3 × 2 × 1 = 120

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
% Execution Aborted
[debug] [1] ?- fact(5,R).
R = 120 ,
```



ملاحظة:

يمكننا القيام بعملية تتبع للكود من خلال التعليمة. trace و سيبدأ البرنامج بعرض الخطوات

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
For built-in help, use ?- help(Topic) . or ?- apropos(Wor
d) .

?-
% c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl
compiled 0.00 sec, 2 clauses
?-
! trace.
!
true.

[trace] ?- fact(3,R).
Call: (10) fact(3, _16078) ? creep
Call: (11) 3>0 ? creep
Exit: (11) 3>0 ? creep
Call: (11) _18778 is 3+ -1 ? creep
Exit: (11) 2 is 3+ -1 ? creep
Call: (11) fact(2, _20288) ? creep
Call: (12) 2>0 ? creep
Exit: (12) 2>0 ? creep
Call: (12) _22858 is 2+ -1 ? creep
Exit: (12) 1 is 2+ -1 ? creep
Call: (12) fact(1, _24068) ? creep
Call: (13) 1>0 ? creep
Exit: (13) 1>0 ? creep
Call: (13) _26938 is 1+ -1 ? creep
Exit: (13) 0 is 1+ -1 ? creep
Call: (13) fact(0, _27848) ? creep
Exit: (13) fact(0, 1) ? creep
Call: (13) _24068 is 1*1 ? creep
Exit: (13) 1 is 1*1 ? creep
Exit: (12) fact(1, 1) ? creep
Call: (12) _20288 is 2*1 ? creep
Exit: (12) 2 is 2*1 ? creep
Exit: (11) fact(2, 2) ? creep
Call: (11) _16078 is 3*2 ? creep
Exit: (11) 6 is 3*2 ? creep
Exit: (10) fact(3, 6) ? creep
R = 6

```

ملخص ما حدث في عملية التتبع (Trace):

بدأ البرنامج بمناداة القاعدة (Call) $fact(3, R)$ ، ودخل في عدة مستويات عودية (10, 11, 12...) حتى وصل لحالة التوقف عند الصفر وتم استخدام المتغيرات: ($_16078$) التي قام البرنامج بتوليدها مؤقتاً ليحجز أماكن في الذاكرة لكل عملية ضرب وطرح وهكذا حتى حساب القيم النهائية وبمجرد وصوله للصفر بدأ البرنامج بالرجوع للخلف لتنفيذ عمليات الضرب وعندما أعطانا (Exit) أي تم نجاح العملية وكانت النتيجة $R = 6$

تصير:

➤ اكتب Predicate يحدد إشارة الرقم (Sign function):

الحل:

إن معرفة إشارة الرقم لا تحتاج إلى تكرار أو تقسيم المشكلة أي لا يحتاج البرنامج أن يستدعي نفسه فقط، نقوم بكتابة الحالات التالية:

$sign(0, 0)$: هذه تسمى حقيقة. (Fact) تخبر البرنامج مباشرة أنه إذا كان المدخل 0 فإن المخرج (الإشارة) هو 0

$sign(X, 1) :- X > 0$: هذه قاعدة. (Rule) تقرأ كالتالي: تكون إشارة X هي 1 بشرط أن يكون X أكبر من 0

$sign(X, -1) :- X < 0$: قاعدة ثانية تقرأ: تكون إشارة X هي -1 بشرط أن يكون X أصغر من 0

```

main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
sign(0, 0).
sign(X, 1) :- X > 0.
sign(X, -1) :- X < 0.

```

○ وعند الاستدعاء نلاحظ:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.4.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl
  compiled 0.00 sec, 3 clauses
?-
  |   sign(5, Result).
Result = 1
```

سيفحص السطر الأول: هل $5 = 0$ لا، فينتقل للسطر الآخر

سيفحص السطر الثاني: هل $5 > 0$ ؟ نعم، إذن سيعطينا $\text{Result} = 1$.

ملاحظة اربسية:

الفرق بين الـ Predicate والـ Function :

● Function: تأخذ مدخلات وترجع مخرجاً واحداً. (Return value)

● الـ Predicate لا يُرجع قيمة، بل يتحقق من صحة العلاقة.

○ إذا قمنا بالاستدعاء $\text{sign}(5, 1)$ سيُرد بـ true لأن العلاقة صحيحة

○ $\text{sign}(5, -1)$ سيُرد بـ false لأن العلاقة خاطئة

○ $\text{sign}(5, R)$ سيحاول إيجاد قيمة لـ R تجعل العلاقة صحيحة فيرجع $R = 1$

تكلما في المحاضرة السابقة عن أنواع البيانات في برولوج وهي (Atoms, Numbers, Complex) وفي هذه المحاضرة سنتكلم عن نوع البيانات الرابع وهو القوائم (Lists)

● **Lists:** هي مجموعة من الكائنات (Objects) ويمكن أن تحتوي على أرقام، ذرات (Atoms)، هياكل معقدة، أو حتى قوائم أخرى

مثال: $[1, \text{ahmed}, X, [], \text{red}(\text{rose})]$

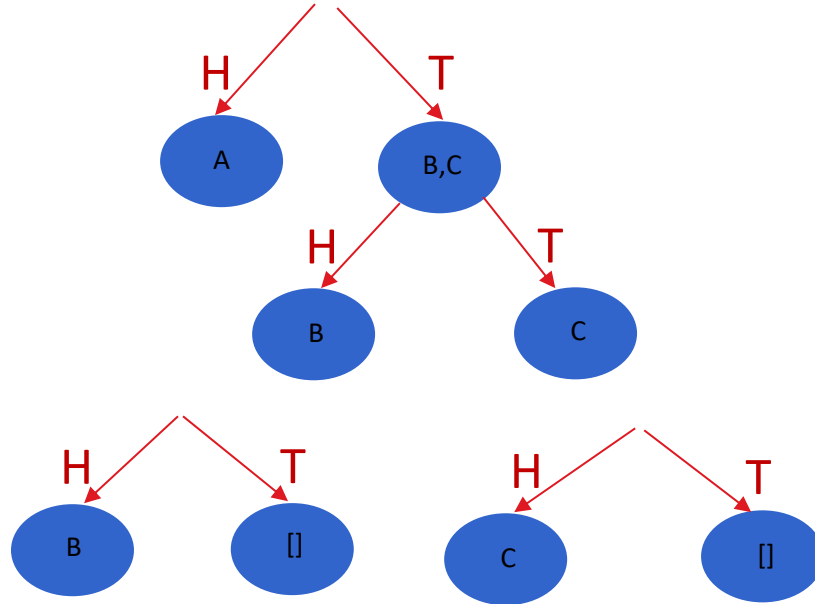
داخلياً، يعالج برولوج القائمة كبنية شجرية أي لا تعتبر القائمة كسطر واحد من العناصر بل يراها كعقد Nodes متداخلة حيث تنقسم كل عقدة إلى فرعين: يسار يحتوي على العنصر الحالي ويمين يحتوي على بقية الشجرة وتعتمد على مفهومين أساسيين يتم الفصل بينهما باستخدام رمز | (البار)

● **الرأس (Head):** وهو العنصر الأول في القائمة فقط في المثال $[X | T]$ ، يمثل $\text{Head } X$

● **الذيل (Tail):** وهو ليس مجرد بقية العناصر بل هو قائمة بحد ذاتها تحتوي على كل ما تبقى بعد Head في المثال السابق، يمثل T هذه القائمة المتبقية.

هذا التقسيم الثنائي [Head | Tail] يسمح لنا بمعالجة البيانات داخل القوائم، حيث يتم اخذ العنصر الأول (Head) للعمل عليه، بينما تمرر بقية القائمة (Tail) للعمليات اللاحقة وصولاً إلى القائمة الفارغة [] التي تعني نهاية البيانات.

مثال: [A, B, C]



تمرين:

➤ اكتب Predicate يقوم بمعرفة أول عنصر في أي قائمة:

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
first(X, [X | _]).
Line: 3
  
```



الحل: نقوم بوضع قاعدة اسمها first تأخذ وسيطين (Parameters):

- **الوسيط الأول:** (X) المكان الذي سيوضع فيه العنصر الأول.
- **الوسيط الثاني:** ([X | _]) القائمة التي نريد فحصها
- هو يرى القائمة كأنها [Head | Tail]
- بما أننا كتبنا X في مكان الرأس، فإنه سيأخذ القيمة ahmed ويضعها داخل X
- بما أننا كتبنا _ في مكان الذيل هذا يعني انه لا يهتمنا ما هي بقية العناصر

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
|
Action (h for help) ? abort
% Execution Aborted
[1] ?- first(X, [ahmed, ali, hassan]).
X = ahmed.
[1] ?-
  
```

○ عند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help

[1] ?- first(ahmed, [ahmed, ali]).
true.

[1] ?-
```

التعريف الثاني:

➤ اكتب Predicate التحقق من أول عنصرين في القائمة:

الحل: الهدف منه الوصول لأول عنصرين حصراً هنا حددنا أن القائمة يجب أن تبدأ بـ X كعنصر أول، يليه Y كعنصر ثان، ثم يأتي التيل T الذي يمثل باقي القائمة (والذي قد يكون فارغاً)

```
main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
first_two(X, Y, [X, Y | T]).
Colourising buffer ... done, 0.00 seconds, 6 fragments
Line: 1
```

إذا كانت القائمة تحتوي على عنصرين فقط سيتم مطابقة العنصر الأول في القائمة مع المتغير X، والعنصر الثاني مع المتغير Y

○ **حالة التيل (Tail):** في هذه الحالة، بما أنه لا توجد عناصر إضافية بعد العنصر الثاني، فإن المتغير T (الذي يمثل باقي القائمة بعد أول عنصرين) سيصبح قائمة فارغة []

○ أما إذا كانت القائمة تحتوي على أقل من عنصرين (عنصر واحد فقط أو فارغة)، فإن التابع سيفشل ويعطي False لأن الشرط الهيكلي الذي وضعه التابع [X, Y | T] يتطلب وجود عنصرين على الأقل ليتمكن من الفصل بينهما

التعريف الثالث:

➤ اكتب Predicate يقوم بالبحث عن عنصر في القائمة:

الحل:

بداية يجب أن نفكر بأن Predicate يجب أن يتكون من قاعدتين:

- **القاعدة الأولى:** (حالة التوقف). `member(X, [X | _]).` وتعني: العنصر X موجود في القائمة إذا كان هو الهيد
- **القاعدة الثانية:** (الاستدعاء الذاتي) `member(X, [_ | T]) :- member(X, T).` تعني: إذا لم يكن X هو الهيد، فتجاهل الهيد وابحث عن X داخل T يستمر هذا حتى يجد العنصر أو تنتهي القائمة

```
main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
member(X, [X | _]).
member(X, [_ | T]) :- member(X, T).
Colourising buffer ... done, 0.00 seconds, 12 fragments
Line: 1
```

عندما قمنا بالاستدعاء ووضعنا حرفاً كبيراً X والذي يعتبر متغيراً Variable في Prolog فنحن نقول أعطنا القيم التي تجعل هذه العلاقة صحيحة

لذا Prolog بحث في القائمة ووجد أن العلاقة تكون صحيحة في ثلاث حالات:

1. إذا كان $X = a$

2. إذا كان $X = b$

3. إذا كان $X = c$

هو أعطانا الاحتمال الأول وتوقف بانتظار أن نسأله عن البقية (بالضغط على space) كما موضح بالصورة:

أما إذا اردنا ان نسأله هل العنصر a موجود في القائمة؟ فيكون الاستدعاء كالتالي:

التمرين الرابع:

➤ اكتب Predicate يقوم بحساب طول القائمة:

الطريقة العادية (Recursion):

تعتمد هذه الطريقة على تأجيل العمليات الحسابية مثلاً عند استدعاء $\text{len}([a, b], X)$.

1. البرنامج يرى $[a, b]$ ، فيقوم بفكها إلى $[a]$ هي H و $[b]$ هي T
2. لا نستطيع حساب X الآن يجب أن نحسب طول التيل $[b]$ أولاً ثم أضيف له 1 هنا نفتح طبقة في الذاكرة
3. ينتقل لـ $[b]$ ، يفكها إلى b و $[]$ ويجب أن نحسب طول $[]$ أولاً وأصبح هناك طبقة ثانية في الذاكرة
4. يصل لـ $\text{len}([], 0)$ الآن عرفنا أن الطول هو 0
5. مرحلة العودة (Unwinding): يبدأ بالرجوع للخلف:
 - الطبقة الثانية $X \leftarrow 0 + 1$: النتيجة 1.
 - الطبقة الأولى $X \leftarrow 1 + 1$: النتيجة 2.

المشكلة: إذا كانت القائمة تحتوي على 100,000 عنصر سيفتح Prolog مئة ألف طبقة في الذاكرة بانتظار العودة وهذا يسبب Stack Overflow

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
len([], 0).
len([_ | T], X) :- len(T, X1), X is X1 + 1.
Buffer saved in file 'main.pl' Line: 2

```

لذلك سنتعلم طريقة المراكم (Accumulator):

1. **من حيث المتغيرات:** تحتاج لثلاثة عناصر: القائمة، المتغير الذي يحسب المراكم Acc، والمتغير النهائي الذي

سيعيد النتيجة X

○ في كل خطوة يقص التابع الهيد من القائمة.

○ مباشرةً، يقوم بزيادة المراكم. $NewAcc \text{ is } Acc + 1$.

○ ثم يستدعي نفسه مع التيل والمراكم الجديد. $len_acc(T, NewAcc, X)$.

2. **شرط التوقف** (الحالة النهائية): عندما تصبح القائمة فارغة [] بصيغة $len_acc([], Acc, Acc)$ ، فهذا يعني أن

الرقم الذي تجمّع في المراكم هو الطول النهائي، فيقوم البرولوج بنقله إلى المتغير X لإعادة استخدامه

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
len(L, X) :- len_acc(L, 0, X).
len_acc([], Acc, Acc).
len_acc([_ | T], Acc, X) :- NewAcc is Acc + 1, len_acc(T, NewAcc, X).
Colourising buffer ... done, 0.02 seconds, 23 fragments Line: 2

```

○ عند الاستدعاء:

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
[1] ?- len([a, b, c], X).
X = 3.

[1] ?-

```

التمرين السادس:

➤ اكتب Predicate يقوم بمطابقة الحروف (a_to_b) :

الحل: هذا التابع يقوم عمله على أن القائمة الأولى تحتوي على 'a' فقط والثانية على 'b' فقط، وأنهما بنفس الطول

يقوم بأخذ H كل قائمة في كل خطوة والتأكد منهما ثم ينتقل للتيل في القائمتين معاً

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
a_to_b([], []).
a_to_b([a | T1], [b | T2]) :- a_to_b(T1, T2).
Colourising buffer ... done, 0.00 seconds, 8 fragments
Line: 3

```

○ عند الاستدعاء:

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
[1] ?- a_to_b([a, a], [b, b,b]).
false.
[1] ?-

```

✍ التمرين السابع:

➤ اكتب تابع إيجاد القيمة العظمى (max) بطريقة المراكم:

- بدلاً من محاولة إيجاد القيمة العظمى عند العودة من التكرار قررنا استخدام المراكم (Acc) ليكون هو الرقم الأكبر حتى الآن في كل خطوة و سنقارن العنصر الجديد بهذا المراكم.
- لا يمكننا أن نبدأ بمراكم فارغ أو بصفر (لأن القائمة قد تحتوي على أرقام سالبة) لذا نأخذ أول عنصر من القائمة ونفترض أنه هو الأكبر حالياً ثم نبدأ البحث في باقي القائمة (التيل).
- أثناء المرور على عناصر القائمة (الهيد والتيل) لدينا حالتين منطقيتين لكل عنصر جديد H:
 - **الحالة الأولى (العنصر الجديد أكبر):** إذا كان الهيد الحالي H أكبر من المراكم Acc الذي نحمله معنا فنستبدل المراكم القديم بـ H ونمرره للاستدعاء التالي
 - **الحالة الثانية (العنصر الجديد أصغر أو يساوي):** إذا كان الهيد الحالي H أصغر من أو يساوي المراكم Acc الذي نحمله نتجاهل H ونحتفظ بـ Acc كما هو، ونمرره للاستدعاء التالي
 - **شرط التوقف:** عند الوصول للنهاية، تكون القيمة الموجودة في المراكم هي فعلياً أكبر قيمة في القائمة بالكامل، لذا نقوم بنقلها إلى المتغير النهائي

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
max([H | T], Max) :- max_acc(T, H, Max).
max_acc([], Max, Max).
max_acc([H | T], Acc, Max) :- H > Acc, max_acc(T, H, Max).
max_acc([H | T], Acc, Max) :- H <= Acc, max_acc(T, Acc, Max).
Buffer saved in file 'main.pl'
Line: 1

```

○ عند الاستدعاء:

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
% C:/Users/Dell/Desktop/telegram downloads/prolog/main.pl
compiled 0.00 sec, 2 clauses
[1] ?-
|   max([5, 12, 8, 20, 3], X).
X = 20

```

➤ التمرين الثامن:

➤ اكتب تابع يقوم بإضافة قيمة معينة لنهاية القائمة:

الحل:

- **شرط التوقف:** إذا كانت القائمة الأصلية فارغة، فإن القائمة الناتجة هي قائمة فيها العنصر X فقط. هذه هي نقطة التوقف

- **الحالة التكرارية:**

إذا كانت القائمة غير فارغة، مثلاً [1, 2, 3] ونريد إضافة 5

نحن لا نستطيع وضع 5 فوراً في الأخير

لذلك نأخذ Head ونحتفظ به ثم نمرر باقي القائمة (Tail) للتابع مرة أخرى ليحاول وضع 5 في آخره (الاستدعاء الذاتي في كل مرة)

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
append_last([], X, [X]).
append_last([H | T], X, [H | T2]) :- append_last(T, X, T2).

```

○ عند الاستدعاء:

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
[1] ?-
|   append_last([1, 2, 3], 4, Result).
Result = [1, 2, 3, 4].
[1] ?-

```

➤ التمرين التاسع والأخير:

➤ اكتب تابع يقوم بعكس القائمة:

الحل:

- **شرط التوقف:** إذا كانت القائمة فارغة فلا يوجد شيء لنعكسه هنا

- القاعدة التكرارية (المنطق):

لكي نعكس قائمة مثل $[a, b, c]$:

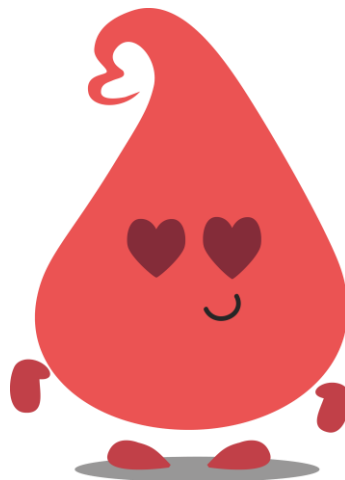
نفصل H وهو a (عن باقي القائمة) T وهي $[b, c]$, نطلب من التابع (بالاستدعاء الذاتي) أن يعكس لنا القائمة $[b, c]$ أولاً والنتيجة ستكون $[c, b]$ وسمينا هذه النتيجة $T1$ والآن لدينا القائمة المقلوبة $[c, b]$ ومعنا H وهو a لكي يكتمل العكس يجب أن نضع a بعد القائمة المقلوبة ليصبح الناتج $[c, b, a]$

ملاحظة: استخدمنا تابع ال `append` الموجود أساساً في اللغة وهو تابع جاهز في Prolog وظيفته الأساسية دمج قائمتين لإنتاج قائمة ثالثة.

```
main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
reverse_simple([], []).
reverse_simple([H | T], R) :-
    reverse_simple(T, T1),
    append(T1, [H], R).
Colourising buffer ... done, 0.02 seconds, 13 fragments
Line: 1
```

○ عند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
[1] ?-
|   reverse_simple([1, 2, 3], Result).
Result = [3, 2, 1].
[1] ?- █
```



انتهت المحاضرة

<The end>